

merrypopins: A Python package for nanoindentation data science

Cahit Acar¹, Anna Marcelissen¹, Hugo van Schrojenstein Lantman¹, and John M. Aiken^{1,2,3}✉

¹ Utrecht University, The Netherlands ² Expert Analytics, Norway ³ University of Oslo, Njord Centre, Norway ✉ Corresponding author

DOI: [10.21105/joss.09933](https://doi.org/10.21105/joss.09933)

Software

- [Review](#) ✉
- [Repository](#) ✉
- [Archive](#) ✉

Editor: Fei Tao ✉

Reviewers:

- [@KennethKwabenaKenney](#)
- [@Manishms18](#)

Submitted: 22 July 2025

Published: 06 October 2026

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

merrypopins is a Python library that streamlines nanoindentation data processing, automated pop-in detection, and statistical analysis of collections of pop-in events. Nanoindentation deforms materials experimentally to characterise their behaviour and quantify mechanical properties from load-displacement data (Oliver & Pharr, 1992, 2004). Experiments performed with a spherical tip can also be used to construct stress-strain curves and, by extension, determine the yield point, defining the transition from elastic to plastic deformation (Kalidindi & Pathak, 2008; Pathak & Kalidindi, 2015). Understanding the onset of plasticity at the microscale matters for applications in engineered materials and earthquake mechanics. A common feature during the loading part of these experiments is a sudden increase of indentation depth at constant force, called a “pop-in”. Recognising pop-ins by hand is labour-intensive and subjective, which makes automated, reproducible detection important.

Statement of need

Pop-ins are difficult to detect because they appear in subtle, intermittent, and varied ways within indentation curves. Analysts have historically picked them by hand from depth vs. time or stress-strain curves, an approach that is subjective, labour-intensive, and inconsistent between observers. Modern nanoindentation machines can perform up to 12 indentations per second (Bruker, n.d.), so the volume of data has outgrown what manual inspection can keep up with.

Pop-ins are linked to dislocation motion in crystalline materials and are considered small-scale analogues of earthquakes (Ispánovity et al., 2022; Sato et al., 2020), following the same kind of statistical patterns, such as power-law distributions in size and in waiting time between events. The first pop-in generally coincides with the onset of plasticity, and the stress at which it occurs depends strongly on indenter tip size and on pre-existing plastic deformation (Morris et al., 2011; Shim et al., 2008), and hence on the apparent yield hardness. A small tip stresses a volume more likely to be dislocation-free, so the material stays elastic up to higher stress, whereas a larger tip samples more volume and yields earlier. Both correcting for this size effect and recovering the underlying statistics require every event along a curve rather than only the first, which is precisely what manual picking scales worst at.

The primary users of merrypopins are students and researchers in materials science, geology, nano-mechanics, and earthquake science, fields in which high-resolution indentation is increasingly used to investigate plastic and fracture processes at the microscale. Despite a growing number of studies targeting pop-ins in load-depth curves, almost all rely on manual inspection or on private scripts whose methods are never disclosed, leaving no accessible, reproducible event detection software to build on. What is needed is an adaptable, open-source tool that works out of the box for non-programmers and stays extensible for power users.

New detection techniques, parameter settings, and visualisation modules can be contributed through the public GitHub repository.

State of the field

Software for instrumented indentation falls into three groups.

First, vendor software shipped with commercial indenters, such as the suite supplied with the Bruker Hysitron platform ([Bruker, n.d.](#)), acquires curves and computes hardness and modulus. It is closed-source, tied to one instrument, and offers no pop-in extraction, so analyses built on it cannot be reproduced outside the vendor's environment.

Second, open-source indentation packages target property extraction rather than event detection. micromechanics ([Brinckmann & Micromechanics Team, 2026](#)) reads a wide range of vendor formats and evaluates hardness and Young's modulus by the Oliver-Pharr method ([Oliver & Pharr, 1992, 2004](#)), but has no pop-in detection layer.

Third, a few tools address pop-ins directly. The closest is the PopIn toolbox ([Mercier, 2023](#)), a MATLAB package that fits the Hertz model to load-displacement curves and plots Weibull or time- and temperature-dependent cumulative distributions of the first or second pop-in. Two things make it a poor foundation for our target audience: it requires a commercial MATLAB licence, and it is scoped to the first and second pop-in of a curve rather than to every event along it. Power-law size and waiting-time statistics need the full catalogue. Machine learning has been applied as well: Kossman & Bigerelle ([2021](#)) trained a convolutional neural network that classifies whole curves as containing a pop-in or not, at roughly 93% accuracy. That answers a different question: whether a curve contains a pop-in, not where each one sits. It also needs labelled training curves, and the code was not released.

merrypopins therefore fills a gap. Its contribution is fourfold: per-point localisation of every pop-in along a curve instead of per-curve classification; four complementary detectors behind one interface, with an explicit cross-method agreement score; a statistical suite that turns located events into the stress-strain, precursor, and temporal quantities that pop-in studies report; and a no-code Streamlit app for users who do not program. All of it is MIT-licensed Python installable from PyPI.

Software design

merrypopins is five modules that pass pandas DataFrames along a pipeline: `load_datasets` parses vendor `.txt` curves and `.tdm/.tdx` metadata, `preprocess` trims setup artefacts and zeroes the depth axis at the detected contact point, `locate` detects pop-ins, `statistics` computes stress-strain curves, yield points, and precursor and temporal statistics, and `make_dataset` runs the first three end-to-end and writes annotated tables with overlay plots. Because every stage consumes and returns a DataFrame, users can stop anywhere, inspect the result, or substitute their own step. This was a deliberate trade-off: a single opaque `analyse()` call would be simpler to document, but researchers need to see and correct the contact-point detection before trusting anything downstream.

Choosing and combining detection methods. Four detectors suit different material systems and experimental circumstances. Savitzky-Golay smoothing suppresses high-frequency noise while preserving curve shape ([Savitzky & Golay, 1964](#)), and Fourier spectral differentiation identifies abrupt discontinuities with minimal parameterisation ([Cooley et al., 1969](#)); both are cheap and interpretable, a good first pass over a large batch. Two unsupervised methods adapt to the data: an Isolation Forest finds anomalies in a multidimensional feature space without labelled instances ([Liu et al., 2008](#)), useful when pop-in magnitudes are unknown beforehand, and a convolutional autoencoder learns representations directly from the curve, catching nonlinear patterns a fixed derivative threshold overlooks ([Malhotra et al., 2016](#)), at higher cost. Rather

than asking the user to pick one, `default_locate` runs the enabled methods and reports all of them: each writes its own boolean column, `popin` is their union, `popin_score` counts how many methods fired at a point, and `popin_confident` requires agreement from at least two. Users who need recall take `popin`; users for whom a false positive costs more than a missed event take `popin_confident`.

No pretrained models. Neither learning method ships with weights. Both are fitted, unsupervised, on the curve being analysed at call time, using local stiffness difference and curvature as features, so no labelled training data is needed and a result depends only on the curve and parameters supplied. All four methods are restricted to the loading portion of the curve, up to maximum load, since unloading artefacts are irrelevant to pop-in analysis. TensorFlow, needed only by the autoencoder, is an optional install extra so the other three carry no heavyweight dependency.

Validating detections. Because pop-in ground truth is itself expert judgement, detections are built to be checked rather than merely trusted: `popin_score` makes cross-method agreement visible, flagging events only one detector saw, and `make_dataset` writes an overlay plot per curve with each method's detections in a distinct colour, so a whole batch can be reviewed visually.

Development approach. The library was built with a tutorial-driven development framework (Aiken, 2020; Aiken et al., 2025; Woods et al., 2022). Rather than starting from architectural specifications, the scientist's process is written as an executable lesson, typically a Jupyter notebook: researchers write function stubs inside a scientific narrative and developers implement them against real usage cases. Scientific usability drives the design.

Research impact statement

`merrypopins` is released for use rather than as a prototype: MIT licensed, installable from PyPI, documented, continuously tested in CI, thirteen tagged releases archived on Zenodo (10.5281/zenodo.21906346), with contributing guidelines and a public issue tracker.

Its near-term significance rests on being usable and checkable by its target community. The repository ships 36 real indentation curves with instrument metadata and rendered overlay plots, five tutorial notebooks reproducing the single-file and multi-file workflows, and a test suite covering all five modules, including synthetic curves with pop-ins of known position and size. Detector behaviour and every documented workflow can therefore be re-run independently. The hosted Streamlit app puts the same pipeline in front of experimentalists who do not program, removing the installation barrier that keeps analysis code out of many indentation laboratories.

It was developed on data from the Utrecht University indentation laboratory to support nanoindentation work in the `SerpRateAI` project and `EPOS-eNLarge`, and is the analysis layer for that ongoing research.

Code availability

The `merrypopins` package can be installed via:

```
pip install merrypopins          # three detectors, lightweight
pip install 'merrypopins[cnn]'   # adds the autoencoder, pulls in TensorFlow
```

The source is on GitHub (<https://github.com/SerpRateAI/merrypopins>), together with the datasets, tutorial notebooks, and test suite described above; contributions are welcome by forking the repository and opening a pull request. The Streamlit app is at <https://merrypopins.streamlit.app/>.

AI usage disclosure

The merrypopins library, its test suite, its documentation, and the original version of this manuscript were written by the authors without generative AI assistance.

During revision in response to peer review, an AI coding assistant was used for three things: drafting prose for the “State of the field”, “Software design”, and “Research impact statement” sections from sources the authors selected and checked, adding type annotations to existing function signatures, and refactoring TensorFlow into an optional install extra. Every resulting change was reviewed, tested, and accepted by the authors. All scientific claims, design decisions, and comparisons with other software are the authors’ own, and the authors take full responsibility for both the software and this paper.

Acknowledgements

This project has received funding from the Norwegian Research Council (SerpRateAI, grant no. 334395) and is supported by EPOS-eNLarge funded by the Dutch Research Council (NWO) Roadmap for large-scale research infrastructure. We would like to thank Alissa Kotowski for fruitful conversations.

References

- Aiken, J. M. (2020). *Science coding vs software development*. Substack. <https://mnky9800n.substack.com/p/science-coding-vs-software-development>
- Aiken, J. M., Jones, G., Yin, X., Abele, A. K., Woods, C., Westaway, R. M., & Bamber, J. L. (2025). 4DModeller: A spatio-temporal modelling package. *Journal of Open Source Software*, 10(106), 7047. <https://doi.org/10.21105/joss.07047>
- Brinckmann, S., & Micromechanics Team. (2026). *Micromechanics: Investigation of nanoindentation and SEM data*. GitHub. <https://micromechanics.github.io/micromechanics/>
- Bruker. (n.d.). *Hysitron TI 990 TribolIndenter brochure*. Bruker. <https://www.bruker.com/en/meta/forms/bns-form-pages/brochures/ni/hysitron-ti-990.html>
- Cooley, J. W., Lewis, P. A. W., & Welch, P. D. (1969). The fast Fourier transform and its applications. *IEEE Transactions on Education*, 12(1), 27–34. <https://doi.org/10.1109/TE.1969.4320436>
- Ispánovity, P. D., Ugi, D., Péterffy, G., Knappek, M., Kalácska, S., Tüzes, D., Dankházi, Z., Máthis, K., Chmelík, F., & Groma, I. (2022). Dislocation avalanches are like earthquakes on the micron scale. *Nature Communications*, 13(1), 1975. <https://doi.org/10.1038/s41467-022-29044-7>
- Kalidindi, S. R., & Pathak, S. (2008). Determination of the effective zero-point and the extraction of spherical nanoindentation stress-strain curves. *Acta Materialia*, 56(14), 3523–3532. <https://doi.org/10.1016/j.actamat.2008.03.036>
- Kossman, S., & Bigerelle, M. (2021). Pop-in identification in nanoindentation curves with deep learning algorithms. *Materials*, 14(22), 7027. <https://doi.org/10.3390/ma14227027>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, 413–422. <https://doi.org/10.1109/ICDM.2008.17>
- Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., & Shroff, G. (2016). *LSTM-based encoder-decoder for multi-sensor anomaly detection*. <https://doi.org/10.48550/arXiv.1607.00148>

- Mercier, D. (2023). *PopIn: "Pop-in" analysis from indentation tests*. GitHub. <https://github.com/DavidMercier/PopIn>
- Morris, J. R., Bei, H., Pharr, G. M., & George, E. P. (2011). Size effects and stochastic behavior of nanoindentation pop in. *Physical Review Letters*, 106(1), 165502. <https://doi.org/10.1103/PhysRevLett.106.165502>
- Oliver, W. C., & Pharr, G. M. (1992). An improved technique for determining hardness and elastic modulus using load and displacement sensing indentation experiments. *Journal of Materials Research*, 7(6), 1564–1583. <https://doi.org/10.1557/JMR.1992.1564>
- Oliver, W. C., & Pharr, G. M. (2004). Measurement of hardness and elastic modulus by instrumented indentation: Advances in understanding and refinements to methodology. *Journal of Materials Research*, 19(1), 3–20. <https://doi.org/10.1557/jmr.2004.19.1.3>
- Pathak, S., & Kalidindi, S. R. (2015). Spherical nanoindentation stress-strain curves. *Materials Science and Engineering R*, 91(1), 1–36. <https://doi.org/10.1016/j.mser.2015.02.001>
- Sato, Y., Shinzato, S., Ohmura, T., Hatano, T., & Ogata, S. (2020). Unique universal scaling in nanoindentation pop-ins. *Nature Communications*, 11(1), 4177. <https://doi.org/10.1038/s41467-020-17918-7>
- Savitzky, A., & Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8), 1627–1639. <https://doi.org/10.1021/ac60214a047>
- Shim, S., Bei, H., George, E. P., & Pharr, G. M. (2008). A different type of indentation size effect. *Scripta Materialia*, 59(1), 1095–1098. <https://doi.org/10.1016/j.scriptamat.2008.07.026>
- Woods, C., Hedges, L., Edsall, C., Brooks-Pollock, E., Parton-Fenton, C., McKinley, T. J., Keeling, M. J., & Danon, L. (2022). MetaWards: A flexible metapopulation framework for modelling disease spread. *Journal of Open Source Software*, 7(70), 3914. <https://doi.org/10.21105/joss.03914>